

Step 1: HTML Parsing → DOM Creation

What happens:

- Browser receives **HTML text** from server
- **HTML Parser** converts text → **DOM Tree**

How it works:

1. **Tokenization:** Break HTML into tags, attributes, text
 - Example: <h1>Hello</h1> → tokens: start tag <h1>, text Hello, end tag </h1>
2. **Node creation:** Each token → element node, text node, attribute node
3. **DOM Tree:** Nodes combined → tree structure

Example DOM:

```
html
├── body
│   └── h1
│       └── "Hello World"
```

Step 2: CSS Parsing → CSSOM Creation

What happens:

- Browser downloads CSS (inline / external)
- **CSS Parser** converts CSS → **CSS Object Model (CSSOM)**

How it works:

1. Tokens → selector, property, value
 - Example: h1 { color: red; font-size: 24px; }
2. CSSOM Tree created → all styling info

Example CSSOM:

```
h1
├── color: red
└── font-size: 24px
```

Step 2: CSS Parsing → CSSOM Creation

What happens:

- Browser downloads CSS (inline / external)
- **CSS Parser** converts CSS → **CSS Object Model (CSSOM)**

How it works:

1. Tokens → selector, property, value
 - Example: h1 { color: red; font-size: 24px; }
2. CSSOM Tree created → all styling info

Analogy:

- CSS = painting and decoration instructions
- CSSOM = the “style plan” of the building

Example CSSOM:

```
h1
├── color: red
└── font-size: 24px
```

Step 3: Render Tree Creation

What happens:

- Browser combines **DOM + CSSOM → Render Tree**
- Only **visible elements** included
- Hidden elements (display:none) ignored

Example Render Tree:

```
body
├── h1 (color:red, font-size:24px)
```

Step 4: Layout (Reflow)

What happens:

- Browser calculates **position, width, height** for each element
- Includes box model: padding, margin, border

Example Layout Table:

Element	x	y	width
h1	0	20	300px

Layout = Reflow

Step 5: Paint

What happens:

- Browser draws **pixels for elements**:

- Text
- Background
- Borders
- Images
- Shadows

Analogy:

- Apply colors, wallpapers, decorations

Step 6: Compositing

What happens:

- Paint layers combined → final screen
- GPU helps render efficiently

Analogy:

- Multiple layers of paintings combined → final display on wall

Step 7: JavaScript Parsing & Execution

JS Parsing

- JS Engine (Chrome → V8) parses JS → AST (Abstract Syntax Tree)

Example AST:

VariableDeclaration

├── Identifier: title

└── FunctionCall: querySelector("h1")

JS Execution

- Execution Context created
- Call Stack manages function calls

Example:

```
document.querySelector("h1").style.color = "blue";
```

- JS updates DOM → triggers **reflow** / **repaint** / **composite** as needed

Step 8: DOM Updates & Rendering

Type of Change	Browser Action
Layout/size change	Reflow → Paint → Composite
Color/background	Paint → Composite
Transform/opacity	Composite only (GPU)

Browser updates screen → User sees the change

Step 9: Continuous Rendering

- Browser monitors:
 - User interactions (scroll, click)
 - Animations
 - DOM/CSS changes
 - Network updates
- Rendering pipeline repeats **only when needed**